Systems programming for Hardware Acceleration

CO-3 ALM-1

K. Vinay Bhargav Reddy
2400031280.

1) Synchronization mechanisms in shared-memory programming.

A) Synchronization ensures that multiple threads/processes access shared data safely without conflicts.

Common mechanisms:

* mutex (lock) - only one thread can access critical section at a time.
* Semaphore - Controls access using a counter.
* Spinlock - Thread waits in a loop until lock is available.
* Barrier - All threads wait until everyone reaches a point.
* condition variables - Threads wait for a condition to be true.

2) Communication in Distributed-memory systems using MPI.

A) In distributed memory, each process has its own memory. Communication happens via message passing.

Types of communication:

* point-to-point

=> MPI_send() - send message

=> MPI_Recv() - receive message.

* collective communication.

=> MPI_BCast() - broadcast data.

=> MPI_Reduce() - combine results

=> MPI_scatter() | MPI_Gather()

3) fork-join execution model en openMP.

*) parallel execution model where a master thread creates multiple threads and later joins them.

Steps:

1. program starts with single thread.
2. At parallel region -> fork into multiple threads.
3. Threads execute tasks simultaneously.
4. At end - join back into single thread.

Diagram flow:

Single Thread -> fork -> parallel Threads -> Join -> single Thread.

4) Role of compiler directives in OpenMP

*) special instructions given to compiler to enable to parallel execution.

purpose:

* control parallelism
* manage threads & workload
* Reduce manual coding complexity.

common directives:

* #pragma omp parallel -> create threads.
* #pragma omp for -> parallelize loops.
* #pragma omp critical -> protect shared data.

18/3/26